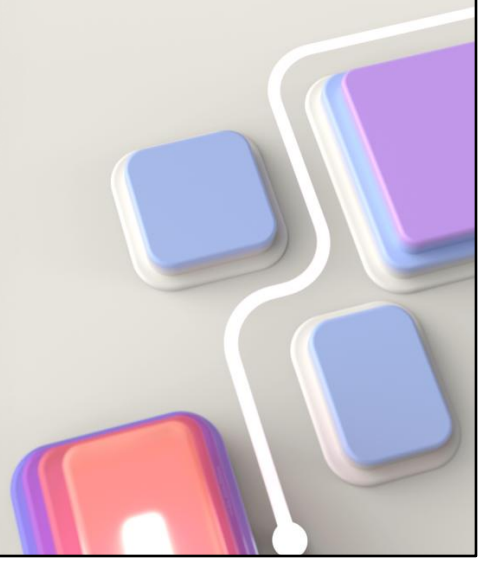
Microsoft

PL-400T00 Microsoft Power Platform Developer

Extending the model-driven app user experience



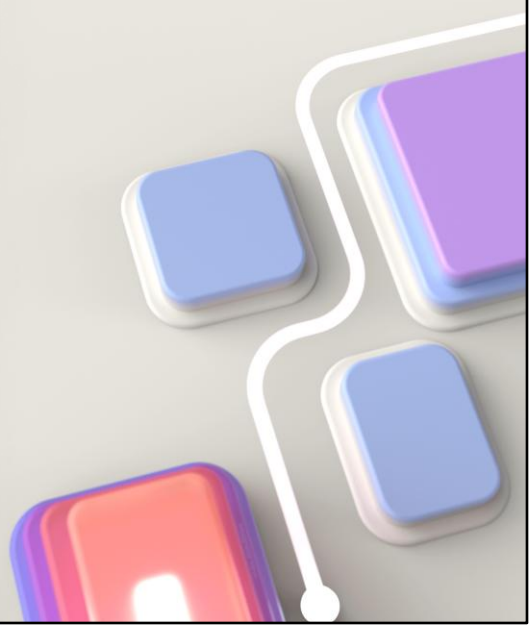
Reference https://docs.microsoft.com/en-us/learn/paths/intro-developing-power-platform/

Modules

- Performing common actions with client script
- Best practices with client script

Modules
- Performing common actions with client script
- Best practices with client script

Module 1: Performing common actions with client script



Learning Objectives

After completing this module, you will be able to:

1. Write client scripts to perform common actions as listed in the module units.

Microsoft Learn module

Performing common actions with client script

https://learn.microsoft.com/training/modules/common-actions-client-script-power-platform

Performing common actions with client script
https://learn.microsoft.com/training/modules/common-actions-client-script-power-platform

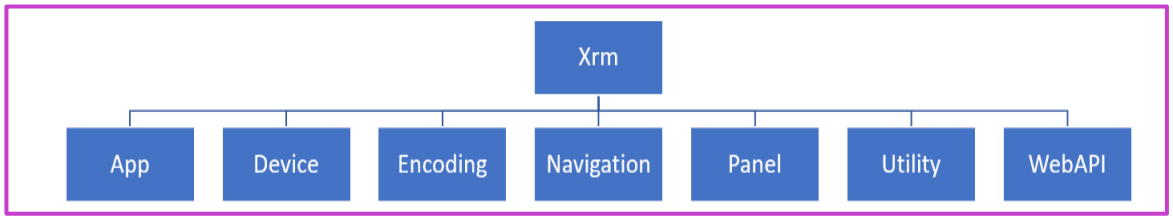
Introduction to client-side scripting

Form events

- Form OnLoad
- Form OnSave
- Column OnChange

Can be used to:

- Get or set column values on the form
- Show and hide user interface elements
- Switch between forms where multiple forms are available for a table
- Open forms and views
- Interact with the business process flow



Client scripting allows you to use JavaScript in Power Apps model-driven apps to implement custom business logic. Client scripting should be used as an alternative when declarative business rules do not meet the requirements. Client scripting runs on a model-driven form in response to form events. The following are the most common events you can register event handlers for:

Form load

Data in a column changed

Form is saved

In addition, a command bar button can be configured to invoke a client script function when pressed.

Some of the common tasks that can be accomplished using client scripting include:

Get or set column values on the form.

Show and hide user interface elements.

Reference multiple controls per column.

Switch between forms where multiple forms are available for a table.

Open forms, views, dialogs, and reports.

Interact with the business process flow control.

Interaction with data, changes of the form content, or modifications of the app behavior should only be implemented using provided client scripting API. While you are writing your logic in JavaScript it is important to note that even though the form is built using standard HTML, direct manipulation of the form content is not supported. Client scripting provides an object model with methods for interacting with the various form components. This ensures your business logic is insulated from any changes in the layout or specific HTML used in the form rendering. It is equally important that you only use documented objects and functions and not any that you may discover, as these can change or even be removed at any time. For more information about supported and unsupported customizations, see [Microsoft Dataverse application patching model - Power Apps](#)in the Dataverse developer guide.

The following is the high-level structure of the client scripting API object model and namespaces:

[!div class="mx-imgBorder"] Screenshot showing all the namespaces in the client scripting API object model: Xrm, App, device, Encoding, Navigation, Panel, Utility, WebAPI.

- **App** - Enables adding event handlers for any app level notifications.
- **Device** - Allows, with consent, access to device content such as image, video, audio, location, and more.
- **Encoding** - Quick access to HTML encode/decode functions.
- **Navigation** - Provides platform-independent navigation functions including open dialogs, forms, files, and URLs.
- **Panel** - Displays the web page represented by a URL in the static area in the side pane, which appears on all pages in the model-driven apps web client.
- **Utility** - Collection of utility functions including access to metadata and several context objects.
- **WebAPI** - Provides properties and functions to use Web API to create and manage records and execute Web API actions and functions.

Another key high-level concept is the context objects that are available either as event handler parameters or can be retrieved using special methods. These context objects help avoid writing code that is hard-wired to a particular form control layout or specific control. The following are the contexts you will work with:

Execution - Defines the event context in which your code executes. The execution context is passed when an event occurs on a form or grid, which you can use in your event handler to perform various tasks such as determine formContext or gridContext or manage the save event.

Form - Provides a reference to the form or to an item on the form, such as, a quick view control or a row in an editable grid, against which the current code executes. Form context is retrieved using getFormContext() method of the execution context or included as an argument when code is executed from a ribbon action. Form context has the following key namespaces within it:

- **data** - This gives access to the data for the table row that is presented on a form. Includes functions like save and refresh. For example, formContext.data.entity.save("saveandnew");
- **ui** - This allows you to manipulate controls on the form such as tabs, sections, and controls. Common tasks include hiding, showing, and making required or not required. For example, formContext.ui.refreshRibbon(true);

Grid - Provides context information to event handlers registered on subgrids on a form.

As you can probably tell from the high-level descriptions that using the client scripting API gives you much flexibility to implement client-side logic. It is important to remember though that logic implemented with client-side scripting is enforced only when the user is using the application. In many cases client scripting must be paired with the server-side implementation to ensure the logic is always enforced regardless of the method used to access the data and functionality. In the rest of this module will dive deeper into how to use client scripting.

Web resources

Upload files to Dataverse

Virtual file system

Use in forms and apps

Use as JavaScript libraries

Types:

- **Script (Jscript)**
- **Webpage (HTML)**
- **Style Sheet (CSS)**
- **Data (XML)**
- **Images**
- PNG
- JPG
- GIF
- ICO

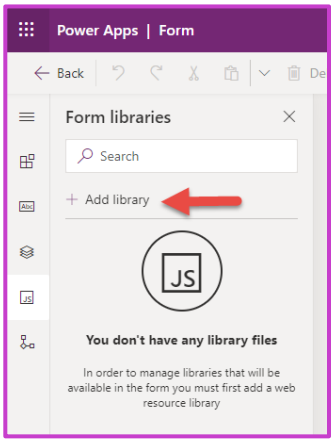
Web resources are a virtual file system that can be referenced by forms and buttons.
You cannot upload files directly but create a web resource and add the file to the web resource

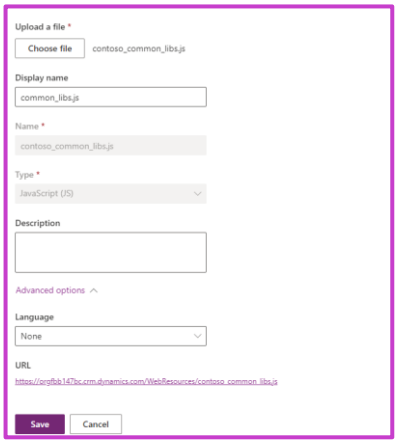
Web resources are primarily used for JavaScript libraries in model-driven apps forms.

https://learn.microsoft.com/power-apps/developer/model-driven-apps/web-resources
https://learn.microsoft.com/power-apps/developer/model-driven-apps/script-jscript-web-resources

Upload scripts

To use client scripting on a form the script must first be uploaded as a JScript web resource





To use client scripting on a form the script must first be uploaded as a script web resource. Script web resources can be used to maintain libraries of client script functions that are written either in JavaScript or TypeScript and that can be accessed from within a model-driven app form or from the command bar ribbon definition. When using TypeScript, it must be transpiled to JavaScript prior to uploading as a web resource.

To upload a script web resource, create a new Forms Library from the form editor.

[!div class="mx-imgBorder"] Screenshot of the Add library dialog.

Make sure to select Script(JScript) as the type.

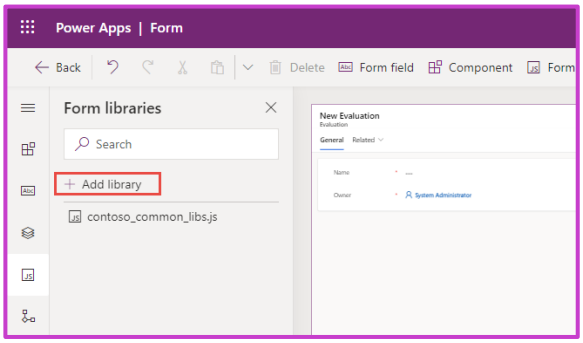
[!div class="mx-imgBorder"] Screenshot showing the add web resource dialog with Type of Script selected.

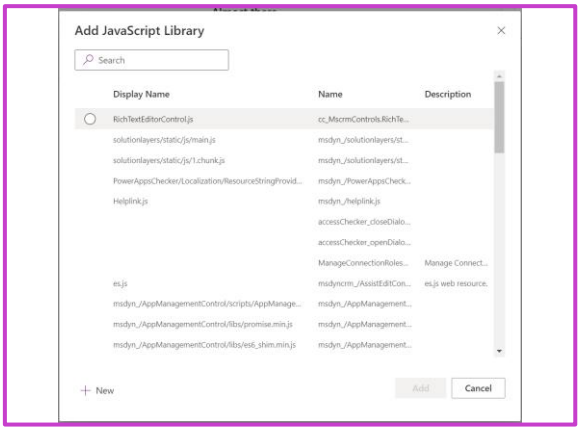
For a table column data to be available for a script to use, the column must be represented by a control on the form. In addition to having to add the column as a control on the form you are at risk of somebody removing it and causing your script to break because the referenced column is no longer available. To ensure that the columns data is always available for your script, you can add the column as a dependency. The following image shows adding the account number column from the account table as a dependency.

[!div class="mx-imgBorder"] Screenshot of adding the account number column from the account table as a dependency.

Using client script libraries

Once configured as a script web resource, client script libraries can be associated with commands and form events





Once configured as a script web resource, client script libraries can be associated with ribbon commands and form events. To associate a script with a form, click on Form libraries in the tool bar and then Add library.

From the Add library dialog, you can either associate an existing uploaded script web resource or create a new one.

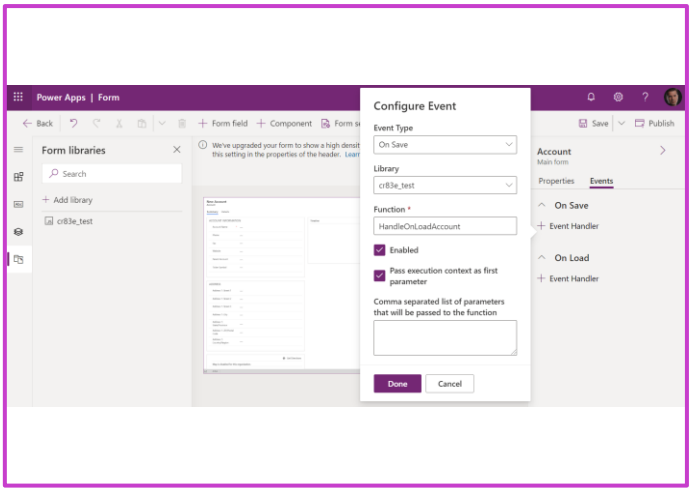
Associating the script library with the form is only required to be done once per script per form regardless of how many event handlers you register on that form.

As you are building your client script logic and need to make changes after the initial upload, you would come back to the script web resource in the solution and upload the new version of the file. After uploading you would need to make sure to publish the script web resource, so the latest changes are used by the app.

Registering via form properties

Registering event handlers using form properties creates a static configuration of event handlers at design time.

- **Form** – This allows you to register **OnLoad** and **OnSave** event handlers.
- **Columns** – This allows you to register an **OnChange** event handler if column data is changed



Manually in Form editor

https://learn.microsoft.com/power-apps/developer/model-driven-apps/clientapi/events-forms-grids

https://learn.microsoft.com/power-apps/developer/model-driven-apps/clientapi/reference/events

https://learn.microsoft.com/power-apps/developer/model-driven-apps/clientapi/walkthrough-write-your-first-client-script

Events handlers

Client scripting logic runs as event handlers for form events. You must register your event handlers to have your logic executed. Registration for common events can be done via the form properties dialog or from code. Some events can only be registered from code. Event handlers can run on multiple forms but must be registered separately for each one.

Registering event handlers using form properties creates a static configuration of event handlers at design time. Each time the form is used the same event handlers will be configured to run each time unlike when you register the event handler with code you can have logic determine what should be registered.

In the form designer you can register event handlers for the following events:

Form - This allows you to register OnLoad and OnSave event handlers.

Tabs - This allows you to register events for each tab on the form for tab state change. Commonly this is used to know if a tab is expanded so you can do something like dynamically load data.

Columns - This allows you to register an event handler if column data is changed.

The following is an example of registering an OnLoad event handler for the account table.

[!div class="mx-imgBorder"] Screenshot showing configuring an event handler in the form designer.

One common pattern is to register an OnLoad handler and then register the remaining event handlers via code in the OnLoad event handler logic. The benefit of this approach is when your logic is used on multiple forms you have do not have to register all the event handlers on each form. The other advantage is if you need to dynamically determine some of the event handlers you can use logic to decide whether to register a handler. For example, you may want to skip registering an event handler for a column if you can determine, when the form is loaded, if this column is read-only or hidden.

Registering using code

Registering event handlers using code is possible for all handlers except OnLoad, which must be registered using configuration

- Create a function that runs in OnLoad of the account form.
- In that function, call addOnChange to register the function to call when the account number column changes.
- Register on the account form properties the OnLoad event handler.

JavaScript

```
function LearnLab_handleAccountOnLoad(executionContext)
{
var formContext = executionContext.getFormContext();

formContext.getAttribute('accountnumber').addOnChange(LearnLab_handleOnChangeAccountNumber)
}
function LearnLab_handleOnChangeAccountNumber(executionContext)
{
var formContext = executionContext.getFormContext();
 formContext.ui.setFormNotification('Check other systems', 'INFO', 'AcctNumber');
}
```


Registering event handlers using code is possible for all handlers except OnLoad, which must be registered using configuration. OnLoad handler can then be used to register other handlers in code. The following is an example of registering an OnChange handler on the account number column in the account table performing the following steps:

Create a function that runs in OnLoad of the account form.

In that function, call addOnChange to register the function to call when the account number column changes.

Register on the account form properties the OnLoad event handler.

function LearnLab_handleAccountOnLoad(executionContext) { var formContext = executionContext.getFormContext(); formContext.getAttribute('accountnumber').addOnChange(LearnLab_handleOnChangeAccountNumber) } function LearnLab_handleOnChangeAccountNumber(executionContext) { var formContext = executionContext.getFormContext(); formContext.ui.setFormNotification('Check other systems', 'INFO', 'AcctNumber'); }

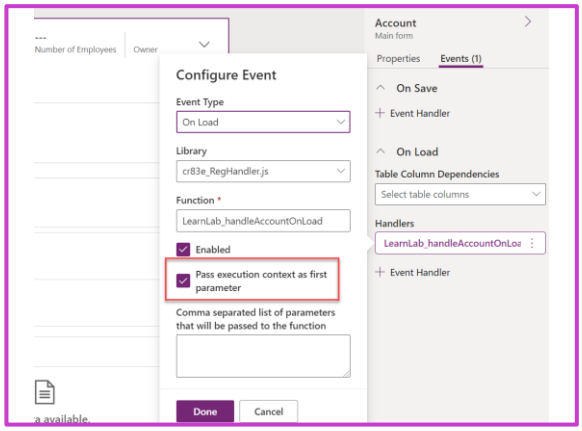
This code will display a form level notification whenever the account number column data changed.

[!div class="mx-imgBorder"] Screenshot showing the form level notification after the custom script logic executed.

There are many other client side events for which you can register handlers.

Execution context

When you register an event handler, you can have the execution context passed as the first parameter.



When you register an event handler, you can have the execution context passed as the first parameter. When you register the event handler using form properties, this is an option that you can enable. The image shows registering an OnLoad handler and enabling execution context.

[!div class="mx-imgBorder"] Screenshot showing how to enable passing the execution context.

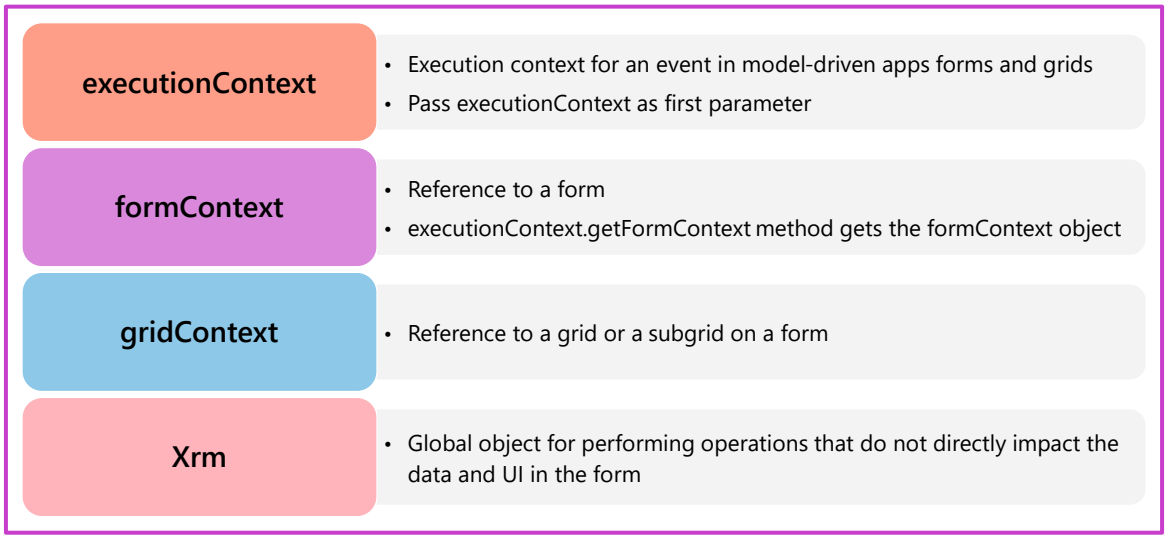
Typically, it's a good idea to always have this option selected when you register an event handler using form properties. When you register an event handler using code this option is automatically selected.

Your function definition that takes execution context as the first parameter would look like the following:

[!div class="mx-imgBorder"] Screenshot of a function highlighting the execution context as the first parameter.

The most common use of the execution context is to retrieve the form and grid contexts. Another useful method on this context is getEventSource. The event source returns a reference to the object that the event occurred. This allows you to write generic handlers that interrogate the event source at run time to find out which control the event occurred on. This can be helpful when you're writing one method that is registered on events for multiple controls instead of registering a separate event handler for each control.

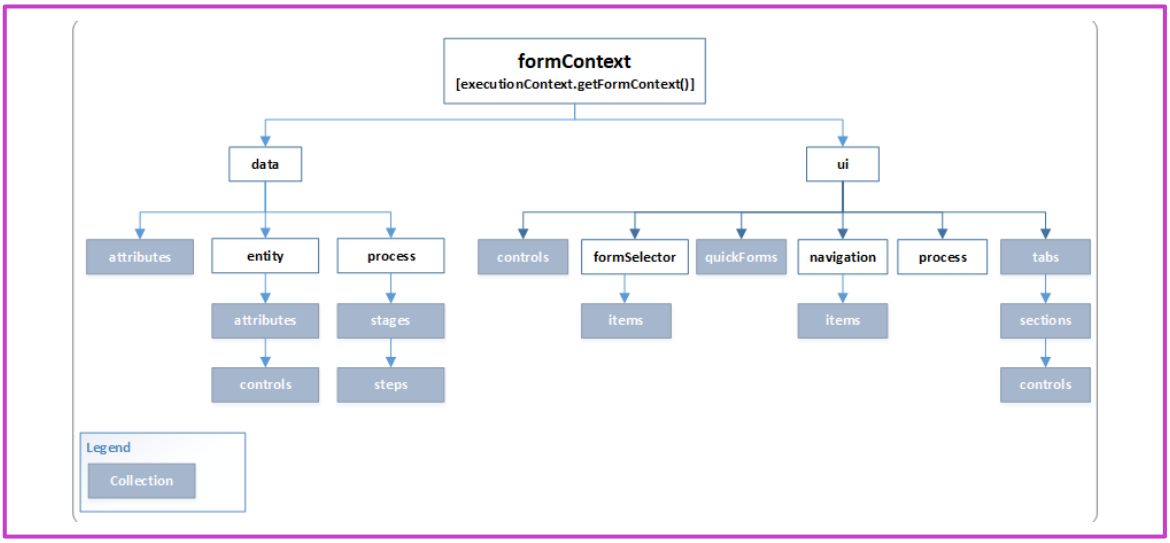
Client API Object model



When creating event handlers and using the client scripting API, you should understand the context objects available and how to use them. The purpose of the context objects is to give you the information about the context your code is executing in. This is so you don't have to hardcode the information in your logic. This allows you to create functions that are more generic and makes your functions less sensitive to the specific layout structure of the UI components you're working with.

https://learn.microsoft.com/en-gb/power-apps/developer/model-driven-apps/clientapi/understand-clientapi-object-model

Form context



The Client API form context (formContext) provides a reference to the form or to an item on the form, such as, a quick view control or a row in an editable grid, against which the current code is executed. You can retrieve the formContext object from the execution context using the getFormContext function.

[!div class="mx-imgBorder"] Screenshot of function using the execution context to get the form context.

Before the introduction of the form context, the global Xrm.Page object was used to represent a form or an item on the form. With the latest version, the Xrm.Page object is deprecated, and you should use the getFormContext method of the passed in execution context object to return reference to the appropriate form or an item on the form. So instead of writing code like the below.

```
var firstName = Xrm.Page.getAttribute("firstname").getValue();
```

Instead write the below code using formContext.

```
var formContext = executionContext.getFormContext(); var firstName = formContext.getAttribute(\"firstname\").getValue();
```

You can learn more about the deprecation of Xrm.Page.

The below diagram is a high-level overview of the properties and methods that are available within the form context:

[!div class="mx-imgBorder"] Diagram of the form context object model.

Form context

The Client API form context (formContext) provides a reference to the form or to an item on the form and replaces Xrm.Page

JavaScript

~~var firstName =~~
~~Xrm.Page.getAttribute("firstname").getValue();~~

var firstName =
formContext.getAttribute("firstname").getValue();

The Client API form context (formContext) provides a reference to the form or to an item on the form, such as, a quick view control or a row in an editable grid, against which the current code is executed. You can retrieve the formContext object from the execution context using the getFormContext function.

Replaces Xrm.Page

[!div class="mx-imgBorder"] Screenshot of function using the execution context to get the form context.

Before the introduction of the form context, the global Xrm.Page object was used to represent a form or an item on the form. With the latest version, the Xrm.Page object is deprecated, and you should use the getFormContext method of the passed in execution context object to return reference to the appropriate form or an item on the form. So instead of writing code like the below.

var firstName = Xrm.Page.getAttribute("firstname").getValue();

Instead write the below code using formContext.

var formContext = executionContext.getFormContext(); var firstName = formContext.getAttribute(\"firstname\").getValue();

You can learn more about the deprecation of Xrm.Page.

The below diagram is a high-level overview of the properties and methods that are available within the form context:

[!div class="mx-imgBorder"] Diagram of the form context object model.

Form types

JavaScript

```
var formContext =
executionContext.getFormContext();

var formType = formContext.getFormType();
```

Form types:

1 Create

2 Update

3 Read Only

4 Disabled

5 Bulk Edit

https://learn.microsoft.com/power-apps/developer/model-driven-apps/clientapi/reference/formcontext-ui/getformtype

Accessing Dataverse column data

Task	Example
Access by name	var nameAttribute = formContext.getAttribute("name"); Assigns the attribute for the **Account Name** column to the **nameAttribute** variable. If attribute is not present on the form, **getAttribute** method returns **null** value.
Access all attributes	var allAttributes = formContext.getAttribute(); Assigns an array of all the attributes in the **formContext.data.entity.attributes** collection to the **allAttributes** variable.

A Dataverse column is represented in the object model as an attribute object.

A Dataverse column is represented in the object model as an attribute object. You can use the getAttribute() method of the formContext object to quickly locate either specific attribute or all attributes present on the form. Each attribute object includes some common methods and other methods depending on the attribute data type.

[!NOTE] The sample code below assumes that the attributes and controls used are present. Most of the methods will return null if objects are not available and usual defensive checks should be used in practice.

[!TIP] Collections returned by Client API methods have some helpful methods that you can use to iterate through the elements. See Collections (Client API reference) in model-driven apps - Power Apps for more information.

[Table was here]

Using attributes

Task	Example
Get the value of an attribute	`var nameValue = formContext.getAttribute("name").getValue();` Assigns the value of the Account Name column to the nameValue variable.
Set the value of an attribute	`formContext.getAttribute("name").setValue("new name");` Sets the value of the Account Name column to “new name”.
Get the currently selected option object in an OptionSet attribute (OptionSet attribute describes the Dataverse Choice column)	`var addressTypeOption = formContext.getAttribute("address1_addresstypecode").getSelectedOption();` Assigns the selected option in the Address Type column to the addressTypeOption variable.
Determine whether an attribute value has changed in the user interface since the form has been opened	`var isNameChanged = formContext.getAttribute("name").getIsDirty();` Assigns a Boolean value that indicates whether the Account Name column value has changed to the isNameChanged variable.

Change whether data is required in a column in order to save a record	`formContext.getAttribute("creditlimit").setRequiredLevel("required");`
	Makes the Credit Limit column required.
	`formContext.getAttribute("creditlimit").setRequiredLevel("none");`
	Makes the Credit Limit column optional.

Determine whether the data in an attribute will be submitted when the record is saved	`var nameSubmitMode = formContext.getAttribute("name").getSubmitMode();` The nameSubmitMode variable value will be either always, never, or dirty text to represent the submitMode for the Account Name column.
Control whether data in an attribute will be saved when the record is saved	`formContext.getAttribute("name").setSubmitMode("always");` The example will force the Account Name column value to always be saved even when it has not changed.
When column level security has been applied to an attribute, determine whether a user has privileges to perform create, read, or update operations on the attribute	`var canUpdateNameAttribute = formContext.getAttribute("name").getUserPrivilege().canUpdate;` Assigns a Boolean value that represents the user's privilege to update the Account Name column to the canUpdateNameAttribute variable.

Demo: JavaScript and OnLoad event

DEMO

Accessing form controls

Task	Example
Access all the controls for a specific attribute	var nameControls = formContext.getAttribute("name").controls.get(); Assigns an array of all the controls for the **name** attribute to the **nameControls** variable.
Access a control by name	var nameControl = formContext.getControl("name"); The first control added to a form for a column will have the same name as the column. Each additional control name will have an index number appended to the name. For example, three controls for the name column will have the names: **name, name1, and name2** respectively.
Access all controls	var allControls = formContext.getControl(); Assigns an array of all the controls in the **formContext.ui.controls** collection to the **allControls** variable.

[Table was here]

Use form controls

Control objects like attribute objects have a common set of methods regardless of the type.

Task	Example
Determine if a control is visible	var isNameVisible = formContext.getControl("name").getVisible(); Assigns a Boolean value to the isNameVisible variable that represents whether the Account Name column is visible.
Hide or show a control	formContext.getControl("name").setVisible(false); Hides the Account Name column.
Get a reference to the attribute for the control	var nameAttribute = formContext.getControl("name").getAttribute(); Assigns the attribute for the control for the Account Name column to the nameAttribute variable.
Disable or enable all controls for an attribute	formContext.getAttribute("name").controls.forEach(function (control, index) { control.setDisabled(true); });
Change the label for a control	formContext.getControl("name").setLabel("Company Name"); Sets the label for the Account Name column to the text Company Name.
Get the parent of a control	var parentSection = formContext.getControl("name").getParent(); Assigns the parent control of the Account Name column to parentSection variable.
Set focus on a control	formContext.getControl("name").setFocus(); Set current input focus to the Account Name column.

Control objects like attribute objects have a common set of methods regardless of the type. They also have specialized methods based on the type of control.

[Table was here]

Use tabs and sections

Each form has collection of tabs.

Task	Example
Show or hide a tab	formContext.ui.tabs.get("general").setVisible(false); Hides general tab.
Change the label for a tab	formContext.ui.tabs.get("general").setLabel("Major"); Sets the label of the general tab to the text Major.
Show or hide a section	formContext.getControl("industrycode").getParent().setVisible(false); Hides section containing the Industry Code column.

Each form has collection of tabs. Each tab has a collection of sections. Each section has a collection of controls. You can programmatically access these elements and use their methods.

[Table was here]

Use entity data

The following table contains methods you can use to get information about the current record.

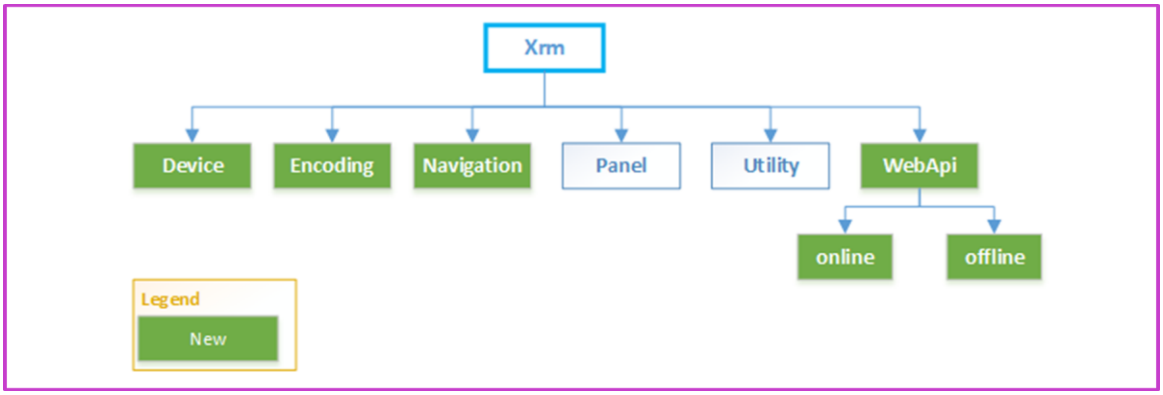
Task	Example
Get the ID of the current record	var recordId = formContext.data.entity.getId(); Assigns a current record unique identifier to the recordId variable. If the form is opened to create a new record null value is returned
Save the current record	formContext.data.entity.save(); Use saveandclose or saveandnew to perform the equivalent actions using the Save & Close or Save & New
Determine whether any data in the current row is changed	var isDirty = formContext.data.entity.getIsDirty(); Assigns a Boolean value that indicates whether any column value on the form has changed to the isDirty variable

The following table contains methods you can use to get information about the current record.

[Table was here]

Introduction to conducting global operations with the client API Xrm object

The Client API provides a globally accessible object (Xrm), which is available for use in your code to perform a variety of activities



The Client API provides a globally accessible object (Xrm), which is available for use in your code to perform a variety of activities. At a high level, the following graphic illustrates each of the properties and methods that are available. For an in-depth overview of this object, see Client API Xrm object.

Xrm.WebApi object

The Xrm.WebApi object provides properties and methods to use the Web API for traditional CRUD operations within a client script.

Method	Description
createRecord	Creates a table row.
deleteRecord	Deletes a table row.
retrieveRecord	Retrieves a table row.
retrieveMultipleRecords	Retrieves a collection of table rows.
updateRecord	Updates a table row.
isAvailableOffline	Returns a Boolean value that indicates whether a table is present in a user's profile and is currently available for use in offline mode.
execute	Run a single action, function, or CRUD operation.
executeMultiple	Run a collection of action, function, or CRUD operations.

The Xrm.WebApi object provides properties and methods to use the Web API for traditional CRUD operations within a client script. The following is a summary of the methods that are available within the Xrm.WebApi object. For more information, see Xrm.WebApi (Client API reference).

Method Description

createRecord

Creates a table row.

deleteRecord

Deletes a table row.

retrieveRecord

Retrieves a table row.

retrieveMultipleRecords

Retrieves a collection of table rows.

updateRecord

Updates a table row.

isAvailableOffline

Returns a Boolean value that indicates whether a table is present in a user's profile and is currently available for use in offline mode.

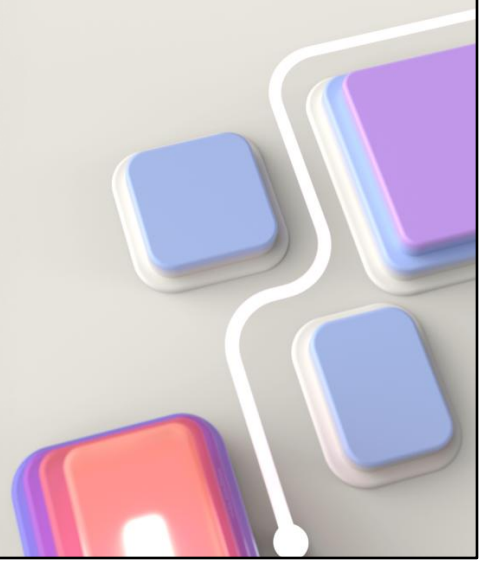
execute

Run a single action, function, or CRUD operation.

executeMultiple

Run a collection of action, function, or CRUD operations.

Module 2: Best practices with client script



This lesson is an add-on for exam coverage purposes

Learning Objectives

After completing this module, you will be able to:

1. Automate business processes using JavaScript/TypeScript API methods.

Microsoft Learn module

Automate business process flows with client script

https://learn.microsoft.com/training/modules/automate-business-process-flow-client-script-power-platform

Automate business process flows with client script – Note this module is badly named and has nothing to do with business process flows

https://learn.microsoft.com/training/modules/automate

-business-process-flow-client-script-power-platform

Client scripting vs. Business rules

JavaScript

- Model-driven app form only
- Operates on columns, sections, and tabs
- Use Web API to access any table/column
- Use with command buttons
- Supports complex logic
- Supports OnSave
- Notifications

Business rules

- Model-driven app form and Dataverse
- Operates on columns only
- Limited to columns on table/form
- Show error next to field
- Events OnLoad and OnChange only
- Recommendations

Business rules is an available feature in model-driven applications that enable users to conduct frequently required actions on a form based on certain requirements and conditions.

While the available framework is robust, some scenarios might exist where business rules can't fully achieve a given requirement. Luckily in these cases, client scripting can narrow the differences between business rules and a more custom requirement. This unit explores some common limitations that users experience with business rules where client script might still be a better solution.

Referencing related data

If you need to reference data that is contained in a related table, such as the address of the primary contact on an account, you'll need to use client script and the Web API.

Logic needs to run on the Form Save event

Business rules only run on form load and on the change of a column. If you need business logic to run On Save, you'll need to use client script to do so.

Complex conditions

If your condition has multiple `and` statements, or requires `or` statements, it might be more efficient to write these types of conditions in client script than through business rules.

Clearing values of form data

Clearing data from a form column isn't easily accomplished by means of a business rule. While there are workarounds that can be achieved by assigning “dummy” columns that always contain NULL values, this is more elegantly accomplished through client script.

Business process flows and JavaScript and Business rules

Actions on a column with a Business rule or with JavaScript performs the same action on the corresponding step in a Business Process flow

- SetVisibility
- SetValue

Calculated columns vs. client script

- Client script are actioned immediately in the user interface.
- Calculated columns are calculated on retrieve, so client changes won’t show until data is refreshed.

Columns are calculated on retrieve, so client changes won't show until data is refreshed. If you want data to update instantly on the form, client script would be the preferred method.

Coding Standards and Best Practices

Define unique Script function names

Unique Function Prefix

JavaScript

```
function MyUniqueName_performMyAction()
{
// Code to perform your action.
}
```


The functions that you write will most likely be loaded into a form with many other libraries. If another library contains a function that has the same name as the one that you provide, whichever function is loaded last is the one that will be defined for that page. To avoid a potential conflict, make sure that you use unique function names. You can use the following strategies if you or your organization does not have one already established:

- Unique Function Prefix - Define each of your functions by using the standard syntax with a consistent name that includes a unique naming convention, as shown in the following example.
- function MyUniqueName_performMyAction(){// Code to perform your action.}
- Namespaced Library Names - Associate each of your functions with a script object to create a kind of namespace to use when you call your functions, as shown in the following example.
- //If the MyUniqueName namespace object isn't defined, create it.if (typeof (MyUniqueName) == "undefined") { MyUniqueName = {}; } // Create Namespace container for functions in this library; MyUniqueName.MyFunctions = {performMyAction: function(){// Code to perform your action.//Call another function in your librarythis.anotherAction(); }, anotherAction: function(){// Code in another function }};

function MyUniqueName_performMyAction() { // Code to perform your action. } //If the MyUniqueName namespace object isn't defined, create it. if (typeof (MyUniqueName) == "undefined") { MyUniqueName = {}; } // Create Namespace container for functions in this library; MyUniqueName.MyFunctions = { performMyAction: function(){ // Code to perform your action. //Call another function in your library this.anotherAction(); }, anotherAction: function(){ // Code in another function } };

When you use your function, you can specify the full name, as shown in the following example.

MyUniqueName.MyFunctions.performMyAction();

If you call a function within another function, you can use the this keyword as a shortcut to the object that contains both functions. However, if your function is being used as an event handler, the this keyword will refer to the object that the event is occurring on.

Coding Standards and Best Practices

Define unique Script function names

Namespaced Library Names

JavaScript

```
//If the MyUniqueName namespace object isn't defined, create it.
if (typeof (MyUniqueName) == "undefined")
{ MyUniqueName = {}; }
// Create Namespace container for functions in this library; MyUniqueName.MyFunctions = {
performMyAction: function(){
// Code to perform your action. //Call another function in your library
this.anotherAction(); },
anotherAction: function(){
// Code in another function } };
```

When you use your function, you can specify the full name, as shown in the following example.

```
MyUniqueName.MyFunctions.performMyAction();
```


The functions that you write will most likely be loaded into a form with many other libraries. If another library contains a function that has the same name as the one that you provide, whichever function is loaded last is the one that will be defined for that page. To avoid a potential conflict, make sure that you use unique function names. You can use the following strategies if you or your organization does not have one already established:

- Unique Function Prefix - Define each of your functions by using the standard syntax with a consistent name that includes a unique naming convention, as shown in the following example.
- function MyUniqueName_performMyAction(){// Code to perform your action.}
- Namespaced Library Names - Associate each of your functions with a script object to create a kind of namespace to use when you call your functions, as shown in the following example.
- //If the MyUniqueName namespace object isn't defined, create it.if (typeof (MyUniqueName) == "undefined") { MyUniqueName = {}; } // Create Namespace container for functions in this library; MyUniqueName.MyFunctions = {performMyAction: function(){// Code to perform your action.//Call another function in your librarythis.anotherAction(); }, anotherAction: function(){// Code in another function }};

function MyUniqueName_performMyAction() { // Code to perform your action. } //If the MyUniqueName namespace object isn't defined, create it. if (typeof (MyUniqueName) == "undefined") { MyUniqueName = {}; } // Create Namespace container for functions in this library; MyUniqueName.MyFunctions = { performMyAction: function(){ // Code to perform your action. //Call another function in your library this.anotherAction(); }, anotherAction: function(){ // Code in another function } };

When you use your function, you can specify the full name, as shown in the following example.

MyUniqueName.MyFunctions.performMyAction();

If you call a function within another function, you can use the this keyword as a shortcut to the object that contains both functions. However, if your function is being used as an event handler, the this keyword will refer to the object that the event is occurring on.

Debugging client script

Nearly every browser has some sort of debugging extension that allows for debugging of client scripts.

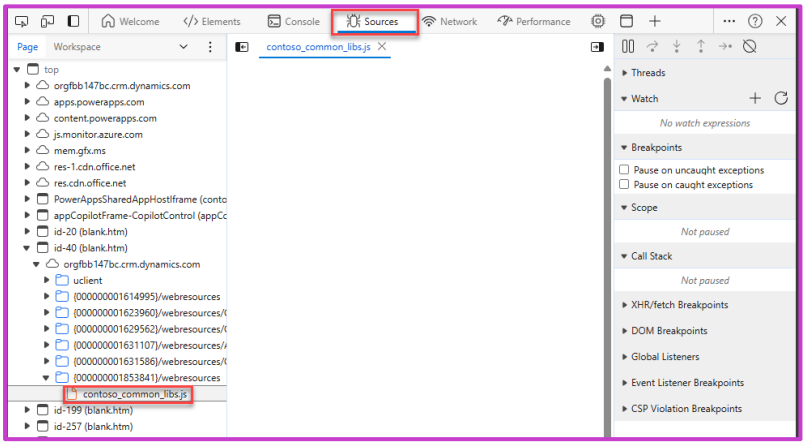
- Microsoft Edge (through F12 Developer Tools)
- Google Chrome (through F12 Developer Tools)
- Mozilla Firefox (using Firebug)
- Apple Safari (using Web Inspector)

Nearly every browser has some sort of debugging extension that allows for debugging of client scripts. The following toolsets are those that are frequently used to perform debugging operations:

- Microsoft Edge (through F12 Developer Tools). For more information, see Using the F12 developer tools guide.
- Internet Explorer (through F12 Developer Tools)
- Google Chrome (through F12 Developer Tools)
- Mozilla Firefox (using Firebug)
- Apple Safari (using Web Inspector)

Viewing script resources

When your model-driven app form page is loaded, all client script libraries are loaded alongside the webpage as individual script resources.



When your model-driven app form page is loaded, all client script libraries are loaded alongside the webpage as individual script resources. Given the volume of script resources that are needed to run a model-driven app, it can be difficult to locate a given file that you might want to debug against. When using debugging tools like Microsoft Edge, we commonly recommend that you note your file name and use the tooling's search capabilities to locate your script files.

Use Fiddler Auto-Responder to replace web resource content

Constantly editing web resources when they are under development can prove to be difficult and time-intensive when you have to republish the files within a solution upon every edit.

Constantly editing web resources when they are under development can prove to be difficult and time-intensive when you have to republish the files within a solution upon every edit. To improve efficiency, consider using a tool like Auto-Responder in Telerik Fiddler to replace content of a web resource with content from a local file, rather than uploading it and republishing each time. Several other third-party tools that also support live editing can be considered. For more information on how to install and configure Fiddler Auto-Responder, see, Script web resource development using Fiddler Auto-Responder.

https://learn.microsoft.com/power-apps/developer/model-driven-apps/streamline-javascript-development-fiddler-autoresponder

Learning Path 4 practice labs

Extending the model-driven app user experience

Lab 6: Client scripting

In this lab, you will implement client-side logic that will use the web API to evaluate the permit type associated with the permit record and use the client scripting API to manipulate the form controls.

You will also customize the command bar to introduce a new lock permit button that will invoke a custom API to perform the lock permit logic. The server-side logic for the lock permit custom API will be implemented later in the course. Right now, you will just add the button and the logic to invoke the custom API.

Summary

- Client scripting provides advanced methods for implementing business logic in your model-driven applications when business rules or other declarative means are not feasible
- Client scripting exposes a robust API that allows you to work with the user interface, modify behavior of the model-driven app, and interact with the data in the underlying Microsoft Dataverse
- JavaScript developers need to be familiar with the scripting techniques specific to model-driven apps such using the client API and scripting best practices applicable to model-driven apps

Modules

- Performing common actions with client script
- Best practices with client script